Joint VNF load balancing and service auto-scaling in NFV with multimedia case study

A.H Ghorab*,1, A. Kusedghi*,2, M.A Nourian*,2, A. Akbari*,3

*Department of Computer Engineering, Iran University of Science and Technology, Tehran, Iran
[1]ah_ghorab@vu.iust.ac.ir, [2]{kusedghi, amin_nourian}@comp.iust.ac.ir, [3]akbari@iust.ac.ir

***Abstract*—The evolution of 5G key enabler technologies, such as SDN and NFV, has brought network operators attention to procure an efficient service delivery mechanism on each possible computing infrastructure poin of presence. Therefore, it is trivial to leverage service management and orchestration functionalities that operate in harmony since they may have significant impact on each others performance. In this paper, we investigate joint load balancing and auto-scaling of service instances being provisioned on computing infrastructure in edge or cloud. We address the necessity and challenges of designing the load balancing algorithm and scale decision making policy, aware of one another, through several practical scenarios. It is demonstrated that changing VNF load balancing method may deteriorate scaling quality of decision-making. Therefore, scaling policy must be defined according to how load balancing unit distributes traffic among VNFs. We use multimedia signalling service delivery in NFV environment as a case study, and aforementioned solution is implemented using management and orchestration of XeniumNFV open-source tool.**

***Index Terms*—5G, NFV, SDN, Load balancing, Service auto-scaling, Edge Computing**

I. INTRODUCTION

The Fifth Generation (5G) network is the promising solution to meet user requirements in terms of data rate, supporting mobility, improving capacity, delay, and so on. Traditionally, the performance of networks heavily rely on the middleboxes that are deployed in the network and input traffic must pass through them according to Service Function Chaining (SFC) concept. The advent of Software-Defined Networking (SDN) has brought the required flexibility in traffic steering by separating control and data plane in order to provide more programmability with global view of the network. On the other hand, Network Function Virtualization (NFV) prescribes decoupling of software components from their respective dedicated hardware to run on inexpensive commodity hardware. In this way, middleboxes or even mobile core network/RAN functions, as well as any service or application, can be instantiated on NFV infrastructure Point of Presence (PoP) in an agile and on demand manner that results in the reduction of CAPEX (Capital Expenditure) and OPEX (Operational Expenditure).

In such NFV and SDN enabled 5G network, Virtual Network Functions (VNFs) can be created/removed rapidly on computing infrastructure in different locations and respective traffic would be steered efficiently towards them. The most important VNFs in this context could be categorized into groups providing or improving: network functionalities and connectivity (e.g., NAT, DNS, Load Balancer), network security (e.g., IDS, Firewall, DPI, Malware Detector), network performance (e.g., Traffic shaping, Rate Limiting, Accelerators), control functions and applications of telecommunication (e.g., EPC components, RAN functions), and elastic services (e.g., IMS, VoIP, web services). The first three types are typically used as a predefined sequence in the network, forming a chain of VNFs called Service Chain (SC). The fourth group includes control functionalities that data plane traffic does not necessarily need to traverse those functions. The last group contains traditional elastic applications that are softwarized and the customer's traffic must be distributed among dynamic number of their instances.

All the above mentioned VNFs may need to be scaled (horizontally/vertically) dynamically in real time to cope with traffic demands. Therefore, for simplification in this paper, we consider auto scaling of elastic services as we could use other types of VNFs since they all have to tackle scaling and load balancing problem in case of fluctuation in traffic demands specially when they are being used in edge computing.

In this paper, we investigate joint load balancing and auto-scaling of service instances being provisioned on computing infrastructure in edge or cloud. We elaborate the necessity of designing the load balancing algorithm and scale decision making policy, aware of one another, through several practical scenarios. Load balancing methods, by themselves, have been used in networks as resource management techniques that may improve the quality of the services being delivered and avoid overload/underload situations. There are also several previous works that aim to devise solutions for auto scaling problem of elastic services, but only a few of them have just mentioned that it is crucial to consider load balancing algorithm while studying auto scaling. The key contributions of this paper are summarized as follows:

- *Auto scaling policy enforcement:* To demonstrate the significance of auto scaling policy, several simple possible policies are implemented and compared on multimedia signaling servers as VNFs with dynamic load balancing.
- *Quality of scale decision making assessment based on different load balancing methods:* We implement some well-known load balancing algorithms with a specific scaling policy on VNFs and we show that even though the

978-1-7281-5937-9/20/$31.00 ©2020 IEEE

policy is fixed, any change in load balancing algorithm may cause scaling engine to make wrong decisions.

- *Evaluation of load balancing-aware auto scaling:* We conduct a set of scenarios to elaborate that even with an appropriate load balancing mehtod, scaling engine would not work properly if the policy is not defined with awareness of load balancing algorithm.
- *Joint VNF load balancing and service auto scaling:* In this paper, we present the joint VNF load balancing and auto scaling problem and utilize the orchestration and monitoring framework of XeniumNFV [1] to run a real case study of elastic SIP servers in SDN/NFV environment. We also discuss about required interactions between NFV units to fully implement this stateful application.

The remainder of this paper is organized as follows. Section II introduces some related works and elaborates our motivation in this paper. In Section III, we present our proposed methods for comprehensive understanding of why scaling and load balancing of VNFs must be aware of one another. We describe our implementation framework design and study the obtained results in Section IV. We conclude the paper in Section V and introduce possible future works.

II. RELATED WORK AND MOTIVATION

Traditionally, server load balancing techniques are used for better resource management of network and computing infrastructure in order to improve the QoS (Quality of Service) of the services being delivered in the network. There are some well-known methods of load balancing that can be categorized as static and dynamic. Static methods use predefined policy as they may have a prediction of traffic and network behavior in contrast to dynamic algorithms that use real-time status of different metrics to adapt their policy. In [2] authors propose a weighted dynamic load balancer for web servers in cloud since most researches introduce weights as decision making factor. Authors in [3] use load and channel agents for dynamic load balancing in cloud. Another research [4] propose a weighted SLA-aware load balancing with load prediction module that takes resource utilizations into account. There are also several researches that study load balancing for specific applications like [5] that targets SIP servers. Similar to them, [6] proposes weighted round robin load balancing for vMMEs in NFV-based EPC of 5G by changing weights adaptively based on latency of the links and number of CPUs allocated. There are also some efforts that do not only balance the load between servers/VNFs, but also try to provide network load balancing [7]. All these approaches assume a fixed number of server or VNFs that is not valid in most cases of current and future networks.

Another possible way to classify previous works is to consider the logical location that load balancing algorithm is being implemented. Some older researches place load balancer physically at the entrance of the network in fixed locations while others take advantage of two emerging technologies (SDN&NFV) to implement load balancer in the control plane on the SDN controller [8] or to instantiate load balancer as a VNF in NFV environment [9]. In some recent works such as [10], load balancing algorithm is implemented as a VNF and SDN controller permits this VNF to redistribute traffic among some other servers/VNFs. In this paper we implement the load balancer as a VNF for two reasons. First, there are numerous types of elastic services being delivered in 5G networks that each one may require its own application specific algorithm. So, developing a load balancing module on the SDN controller for each service type would cause tremendous overhead and SDN controller would become the performance bottleneck of the network. Second, by leveraging facilities offered by NFV to add/remove VNF instances dynamically and using container-based VNFs, we can create load balancer VNFs, any time, wherever needed, and give over the task of steering traffic to SDN controller for them.

In addition to load balancing, scaling could be a promising approach to cope with fluctuation of traffic demands without degradation of QoS. But since, increasing of resources assigned to an elastic service imposes extra cost, we should avoid unnecessary scales in/out or up/down. In this paper we consider horizontal scaling of VNFs since in many real world scenarios vertical scaling is not applicable because of distributed computing resources. Also, by taking advantage of containers, the overhead of creating new instances is considerably reduced.

There are significant works on VNF auto-scaling. Some of them try to select proper metrics for scale decision making and some others try to predict changing pattern of these metrics to proactively scale the service avoiding overload situation. In [11] authors aim to predict resource requirements of different components of IMS VNF by monitoring and learning states of other components due to the directed links between them for traffic exchange. z-Torch [12] is NFV orchestration and monitoring solution that aims to improve QoD such as scaling by analyzing Key-Performance indicators and their affinity. There are also papers devising solutions for scaling of service chains, for example [13] proposes scaling scheme for geo-distributed service chains of mobile core networks and IMS. An extensible framework for elastic service chains in 5G networks is introduced in [14] using open source projects for lifecycle management and failure recovery.

VNF load balancing and auto-scaling are also considered as main solutions for overload control in NFV and 5G networks. Authors in [15] use traffic throttling in three layers of VNF, host and network as a complementary method for overload control. Another recent work [16] suggests that in case of CPU contention regular throttling may not be suffice. Although scaling can resolve the problem in general, but during traffic spikes it does not work well. We believe, if load balancing and scaling solutions work aware of each other they can effectively avoin over/underload. In this way, we can consider them as states of a finite automata for overload control of elastic services that is depicted in Fig. 1.

There are some previous researches that consider both VNF load balancing and scaling but they somehow use them

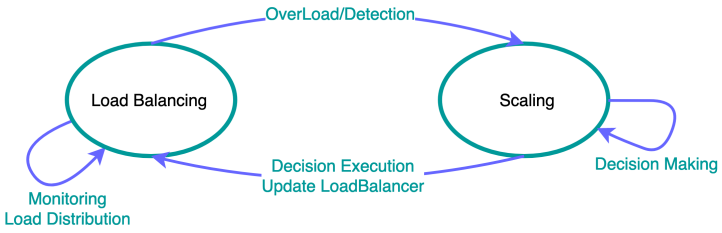


Fig. 1: Finite state automata model for LB and scaling problem.

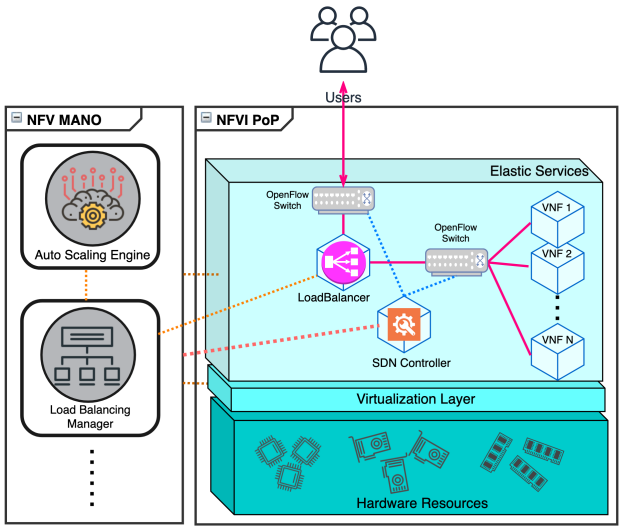


Fig. 2: Joint VNF load balancing and auto-scaling architecture

completely independent of one another. For instance, [17] proposes an architecture supporting autonomic NFV services with SDN, monitoring and demand prediction solutions and uses VNF auto-scaling for its CDN use case but it doesn't study the effect of load balancing method and scaling policy together on the system. Also the network and compute-aware orchestrator of multimedia services [18] considers both these functionalities for SIP servers without a joint approach and does only focus on load balancing. The only work that has investigated VNF load balancing and auto-scaling in a unified manner may be [19] that calculates a very simple score per VM and uses it for scaling algorithm and load balancing algorithm seperately and does not tune any of them to find out the effects on the whole functionality. In Fig. 2, the logical components involved in a joint VNF load balancing and auto-scaling of SDN/NFV environment are shown. To elaborate how interactions are made through this structure, we present a sample sequence diagram as workflow (Fig. 3) for our implementation framework that is described in the next section.

In some recent works a new paradigm of orchestrating elastic services is introduced. VNF load balancing and auto-scaling are used to increase availability and reliability of services in 5G networks that are generally considered as

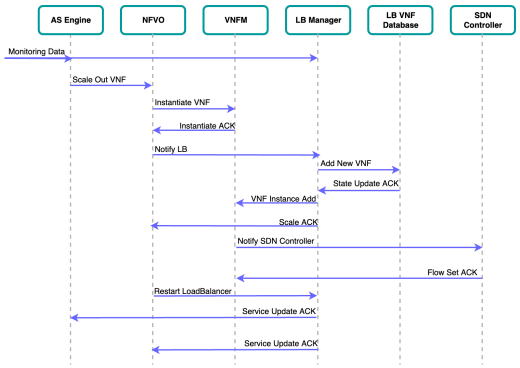


Fig. 3: Sequence diagram of joint VNF load balancing and auto-scaling

dependability attributes. Failure recovery and detection are also important in this context but aforementioned solutions have positive impact on these attributes as well. In [20], the availability levels defined for telecom services are considered to dimension bottlenecks of virtualized mobile core EPC-IMS based. The relation of our dependability is comprehensively discussed in [21]. We previously discussed VNF placement problem in [22] for IMS, so in current work we only focus on Auto-scaling and load balancing.

III. Joint of Scaling Policy and Load Balancing

The main purpose of scalability and load balancing is to reach the desired service level agreement (SLA) by considering the cost of resource allocation . Allocating more resources is only tolerable if it prevents SLA violation. This paper gives a good insight about this problem and identifies the key factors which affects decisions to create more instances. The previous works have addressed the scaling and load balancing problems seperately. In this paper the problems of determining scaling policy and load balancing algorithm are considered together that should be solved simultaneously. It should be noted that the feasibility of this idea is tightly coupled to virtualization and softwarization (SDN and NFV).

We believe the following three factors contribute to the creation of new VNFs and should be considered to reach the optimum point that guarantees the desired QoS and cost efficiency.

- **The policy of scaling:** The first factor affecting the number of VNF instances to maintain service quality is the scaling policy. The number of instances for a particular service may differ if the scaling policy is based on resource utilizations (such as CPU and RAM), traffic rate and etc.
- **Impact of LB algorithm on scaling:** The discussion about load balancing algorithm and scaling problem contains important points that have not received much attention before. We beleive that the load balancing algorithm affects the number of VNF instances. It is also possible that adding another instance, due to inappropriate load balancing, may not improve the poor QoS. To prove this

claim, we use two well-known LB algorithms, Round-Robin (RR) and Weighted Based Distribution (WBD). The RR algorithm is selected as an example of a set of LB algorithms that do not consider resource capacity. RR load balancing is a simple way to distribute requests across a group of instances in turn and finally go back to the top of the list and repeat again. The WBD algorithm is selected as a LB algorithm that is aware of resource capacities and chooses appropriate weights for each instance. The weight of each instance is determined according to the equation (1).

$$w_i = \frac{Res_i}{\sum_{j=1}^{n} Res_j} \tag{1}$$

In this equation, Res_i is the amount of resource for instance i and w_i is the weight given to this instance in the LB algorithm. In WBD algorithm, more volume of traffic would be forwarded to a server with larger weight. Scaling policy is also set with respect to CPU and memory utilizations. In NFV management and orchestration, various parameters are monitored and a new VNF is created automatically if the cost function of those parameters exceeds a specified threshold. To this end, this action is performed by following equation (2).

$$\alpha \times U_{cpu} + \beta \times U_{mem} > th \tag{2}$$

In the above equation, U_{cpu} and U_{mem} are CPU and memory utilizations of VNF. Also, α and β are coefficients that can be changed upon application type.

- **Matching between scaling policy and LB algorithm:** Another factor is the harmony between the scaling policy and the LB algorithm. This issue, which is in line with the previous one, describes that not only the LB algorithm is effective in the scaling problem but also the matching between LB and the scaling policy is very important and must be precisely considered. For example, the ultimate goal is to guarantee the expected QoS with least number of instances that can't be achieved if the weight of instances in the WBD algorithm are chosen based on CPU capacity while the scaling policy is based on the amount of used RAM.

IV. EVALUATION

A. Experimental Setup

In this paper, SIP application is used to illustrate the joint problem of LB and scaling. Fig. 4 is an image of the XeniumNFV environment in which the designed scenarios are implemented. SIPp is used as a UAC and UAS to generate and receive traffic in the SIP scenario. SIPp was also configured to increase the call rate by 20 cps for every 20 seconds untill the maximum limit of 300 calls. Asterisks and OpenSIPs were used as SIP servers in the experiments. The Kamailio load balancer was used to distribute the traffic load between the SIP servers. All of these components were containerized and put together in the XNFV framework. XNFV is responsible

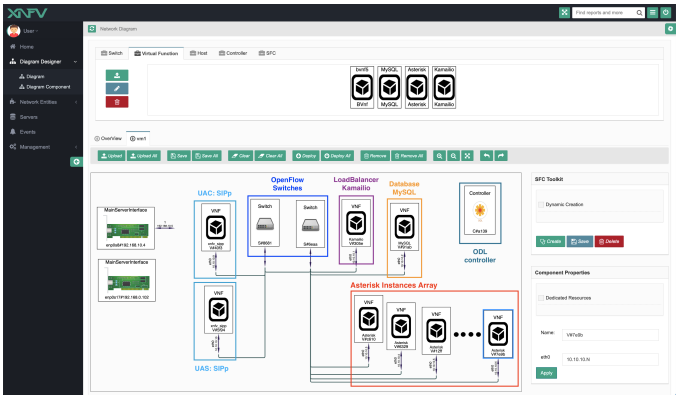


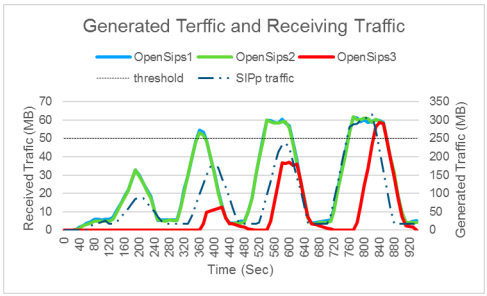
Fig. 4: Implemented SIP scenario in XNFV framework.

for supervising all of these containers and performing the described scaling policy that is designed as an event.

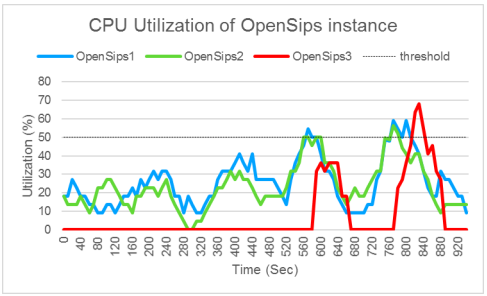
B. Results and Discussion

This section explains the design of experiments and results to confirm how scaling policy and LB algorithm have influence on one another which is thoroughly discussed in section III. Table I lists the LB algorithms, scaling policies, and resources of each SIP server used for each factor.

- **The policy of scaling:** Choosing a suitable scaling policy is critical in determining the number of VNFs. According to table I, two OpenSips servers with the same specification were created and the relative value algorithm was chosen for load balancing to show the effect of this factor on number of scaling. A new instance will be created (or deleted) as soon as the receiving traffic in the SIP server exceeds (or fall short of) 50 MB. Fig. 5a shows the times of creating/removing of the new instance. The load generated by SIPp is also shown. In another test, the scaling policy is based on CPU utilization. Scale out/in is only triggered when the CPU utilization reaches 50%. Fig. 5b shows that in this case, the scaling operation is performed twice, which is less than the previous test.
- **Impact of LB algorithm on scalability:** According to Section III, two experiments were performed with two different LB algorithms. α, β and the threshold value (th) are set to 0.2 and 0.8 and 45%, respectively. Fig. 6a,6b,6c,6d illustrate the resource utilization of each SIP server. Because the RR algorithm doesn't consider the capacity of SIP servers, Asterisk3 resource utilization increases. So the new instance will be created even though Asterisk1 and Asterisk2 are not fully utilized yet. Whereas the weighted LB algorithm distributes the requests proportional to the capacity of resources of each server. So, the scaling policy is never triggered and therefore no new instance is created. This issue becomes even more important when the performance parameters such as response time deteriorates despite of adding a new instance. The response time for each test is shown in Fig. 6e. In RR algorithm, because asterisk3 has fewer



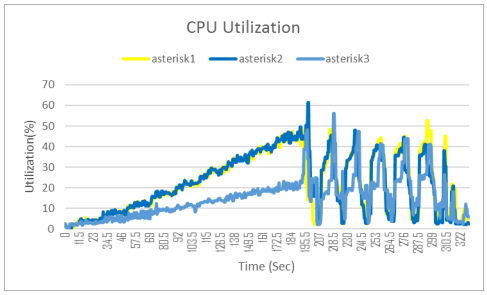
(a) Generated traffic by SIPp - Received traffic at servers



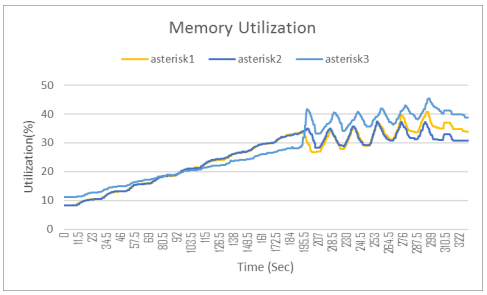
(b) CPU utilization of SIP servers.

Fig. 5: Scaling with different policies (experiment 1).

Weighted LB

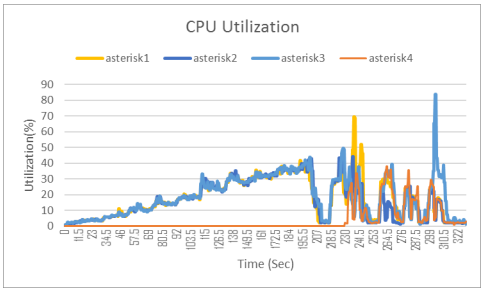


(a) CPU utilization.

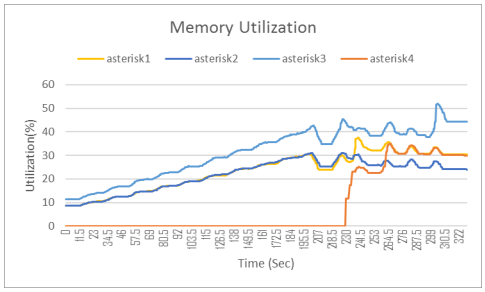


(b) Memory utilization.

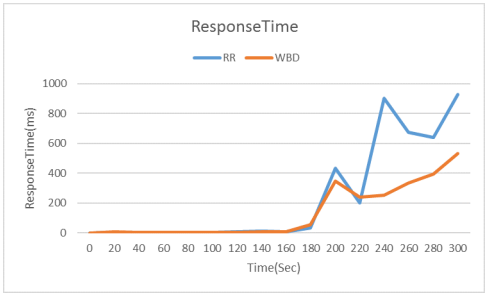
Round-Robin LB



(c) CPU utilization.



(d) Memory utilization.



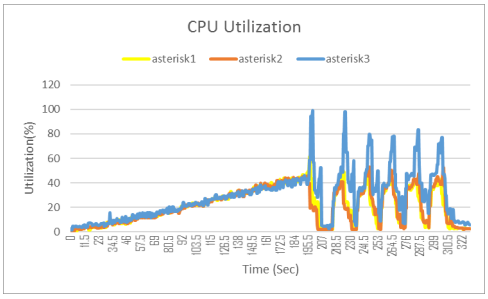
(e) Impact of LB on system response time.

Fig. 6: Resource utilization and performance evaluation in experiment 2.

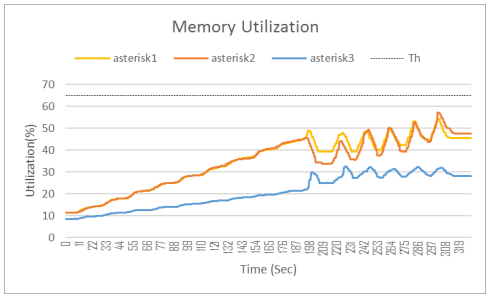
TABLE I: Specification of SIP server resources and algorithms used in each test

Effective factor	LB Algorithm	Scaling Policy	SIP Server resource
Impact of Scaling policy	RR	1.CPU utilization 2.Rx Traffic	OpenSips1: 1vCpu,500MB Ram OpenSips2: 1vCpu,500MB Ram
Impact of LB Algorithm	1.RR 2.WBD (W was set basedon asterisk resources)	α*Ucpu+β*Umem>th	Asterisk1: 1vCpu,400MB Ram Asterisk2: 1vCpu,400MB Ram Asterisk3: 0.5vCpu,200MB Ram
Impact of matching between scaling policy and LBalgorithm	WBD (W was set based on Memory)	1.Memory utilization 2.CPU utilization	Asterisk1: 2vCpu,400MB Ram Asterisk2: 2vCpu,400MB Ram Asterisk3: 1vCpu,800MB Ram

Scaling based on memory

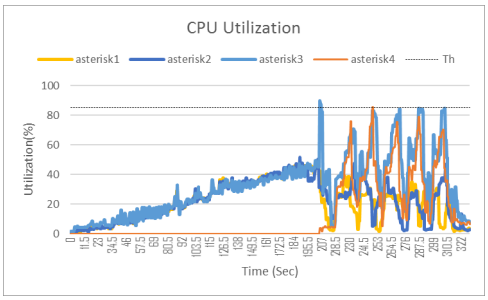


(a) CPU utilization.

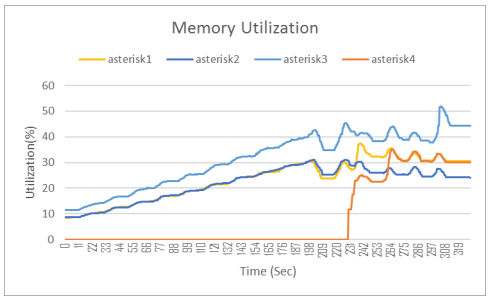


(b) Memory utilization.

Scaling based on CPU



(c) CPU utilization.



(d) Memory utilization.

Fig. 7: Resource utilizations in experiment 3.

resources and it is fully utilized, the response time is higher than the weighted LB experiment. Another notable point in this chart is that although the response time has decreased slightly with the addition of the new source, it still performs worse than the weighted LB because the RR algorithm does not distinguish between instances and sends the requests to Asterisk3 in turn. Therefore, the effect of the LB method on scaling is demonstrated. Also, It can be realized that adding resources does not necessarily lead to better performance.

- **Matching between scaling policy and LB algorithm:** This part emphasizes that a proper load balancing algorithm is not necessarily enough without adaptation with scaling policy. The weighted LB algorithm outperforms RR algorithm because of resource capacity consideration that is thoroughly discussed before. In this section, we show that the inconsistent scaling policy degrades the performance of the system with a fixed LB algorithm (WBD algorithm). We set the weights of the SIP servers to 0.4, 0.4 and 0.2 (proportional to their CPU capacity). In the first test, the scaling policy is based on CPU utilization. A new instance is created if the CPU utilization exceeds 85%. Fig. 7c and Fig. 7a show the CPU and memory utilizations of all three SIP servers. Asterisk3 exceeds the specific threshold due to its lower capacity. Consequently, a new instance is created and added to instance pool. Therefore, this factor leads to maintain the QoS. In the second test, the scaling policy is based on memory utilization and a new instance is created if the memroy utilization exceeds 65%. In this case, none of the SIP servers reach the threshold (Fig. 7b)even though CPU utilization of the Asterisk3 is high and must be scaled (Fig. 7a). In fact, due to incorrect scaling policy, no new instance is created even if it is obvious that more resource is needed.

V. CONCLUSION AND FUTURE WORK

In this paper, through several real scenario implementations (multimedia signalling), we demonstrated that load balancing among VNF instances of an elastic service and auto-scaling of the service must be designed aware of one another in order to improve the quality of decision making which results in QoS improvement and efficient resource management. This procedure is trivial in case of leveraging NFV in different locations of 5G networks such as RAN and mobile core network, edge and cloud networks. We use well-known load balancing methods (RR and weighted RR) and some Key-performance indicators for horizontal scaling policy. For this pupose, We developed an automated orchestration framework of NFV using XeniumNFV SDN/NFV open source testbed.

In future, we aim to devise a novel joint load balancing and auto-scaling solution with a hybrid reactive and proactive approach using machine learning and solve the joint optimization problem by taking into account cost and performance constraints.

REFERENCES

[1] A. Kusedghi, A. Ghorab, and A. Akbari, "Xeniumnfv: A unified, dynamic, distributed and event-driven sdn/nfv testbed," in *2018 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, Dec. 2018, pp. 320–326.

[2] S. H. K. S. L. Chen, Y. Chen, "Clb: A novel load balancing architecture and algorithm for cloud services," *Computers and Electrical Engineering*, vol. 58, pp. 154–160, Feb. 2017.

[3] D. J. A. Singh and M. Malhotra, "Autonomous agent based load balancing algorithm in cloud computing," in *Procedia Computer Science*, vol. 45, Jan. 2015, pp. 832–841.

[4] Chung-Cheng Li and Kuochen Wang, "An sla-aware load balancing scheme for cloud datacenters," in *The International Conference on Information Networking 2014 (ICOIN2014)*, Feb. 2014, pp. 58–63.

[5] S. M. B. A. Akbar and A. Sattar, *A Comparative Study on Load Balancing Algorithms for SIP Servers*. New Delhi: Springer, Feb. 2016, pp. 79–88.

[6] V. Nguyen, K. Grinnemo, J. Taheri, and A. Brunstrom, "On load balancing for a virtual and distributed mme in the 5g core," in *2018 IEEE 29th Annual International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC)*, Sep. 2018, pp. 1–7.

[7] M. Thai, Y. Lin, and Y. Lai, "A joint network and server load balancing algorithm for chaining virtualized network functions," in *2016 IEEE International Conference on Communications (ICC)*, May 2016, pp. 1–6.

[8] X. T. W. Chen, Z. Shang and H. Li, "Dynamic server cluster load balancing in virtualization environment with openflow," *International Journal of Distributed Sensor Networks*, vol. 11, no. 7, pp. 531–538, 2015.

[9] W. Zhang, T. Wood, and J. Hwang, "Netkv: Scalable, self-managing, load balancing as a network function," in *2016 IEEE International Conference on Autonomic Computing (ICAC)*, July 2016, pp. 5–14.

[10] S. J. Y. Go, R. Guinto, C. A. M. Festin, I. Austria, R. Ocampo, and W. M. Tan, "An sdn/nfv-enabled architecture for detecting personally identifiable information leaks on network traffic," in *2019 Eleventh International Conference on Ubiquitous and Future Networks (ICUFN)*, July 2019, pp. 306–311.

[11] R. Mijumbi, S. Hasija, S. Davy, A. Davy, B. Jennings, and R. Boutaba, "Topology-aware prediction of virtual network function resource requirements," *IEEE Trans. on Network and Service Management*, vol. 14, no. 1, pp. 106–120, Mar. 2017.

[12] V. Sciancalepore, F. Z. Yousaf, and X. Costa-Perez, "z-torch: An automated nfv orchestration and monitoring solution," *IEEE Trans. on Network and Service Management*, vol. 15, no. 4, pp. 1292–1306, Dec. 2018.

[13] J. Duan, C. Wu, F. Le, A. X. Liu, and Y. Peng, "Dynamic scaling of virtualized, distributed service chains: A case study of ims," *IEEE Journal on Selected Areas in Communications*, vol. 35, no. 11, pp. 2501–2511, Nov. 2017.

[14] A. M. Medhat, G. A. Carella, M. Pauls, and T. Magedanz, "Extensible framework for elastic orchestration of service function chains in 5g networks," in *2017 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, Nov. 2017, pp. 327–333.

[15] D. Cotroneo, R. Natella, and S. Rosiello, "Nfv-throttle: An overload control framework for network function virtualization," *IEEE Trans. on Network and Service Management*, vol. 14, pp. 949–963, Dec. 2017.

[16] R. N. D. Cotroneo and S. Rosiello, "Overload control for virtual network functions under cpu contention," *Future Generation Computer Systems*, vol. 99, pp. 164–176, Oct. 2019.

[17] L. Velasco, R. Casellas, and S. Llana, "A control and management architecture supporting autonomic nfv services," *Photonic Network Communications*, vol. 37, no. 1, pp. 24–37, Feb. 2019.

[18] R. Moreira, F. de Oliveira Silva, P. Frosi Rosa, and R. Aguiar, "A flexible network and compute-aware orchestrator to enhance qos in nfv-based multimedia services," in *2018 IEEE 32nd International Conference on Advanced Information Networking and Applications (AINA)*, May 2018, pp. 512–519.

[19] R. Poddar, A. Vishnoi, and V. Mann, "Haven: Holistic load balancing and auto scaling in the cloud," in *2015 7th International Conference on Communication Systems and Networks (COMSNETS)*, Jan. 2015, pp. 1–8.

[20] W. Abderrahim and Z. Choukair, "Dependability integration in cloud-hosted telecommunication services," *IEEE Trans. on Dependable and Secure Computing*, pp. 1–1, 2018.

[21] A. J. Gonzalez, G. Nencioni, A. Kamisiński, B. E. Helvik, and P. E. Heegaard, "Dependability of the nfv orchestrator: State of the art and research challenges," *IEEE Communications Surveys Tutorials*, vol. 20, no. 4, pp. 3307–3329, Fourthquarter 2018.

[22] A. Kusedghi, Z. Bagherabadi, and A. Akbari, "An ims-aware vm placement in cloud environment," in *2018 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, Dec. 2018, pp. 327–334.